LONDON METROPOLITAN UNIVERSITY



Islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS4051NI Fundamental of Computing

70% Individual Coursework

Submission : Final Submission

Academic Semester: Summer Semester 2025

Credit: 15 credit semester long module

Student Name: Suhani Singh

London Met ID: 24058840

College ID: NP01AI4S250020

Assignment Due Date: Wednesday, August 27, 2025

Assignment Submission Date: Wednesday, August 27, 2025

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded

TURN IT IN:

Similarity Check

Islington College

Document Details

Submission ID
trn:oid:::2945:20366126

Submission Date
Aug 27, 2025, 10:53 AM GMT+5:45

Download Date
Aug 27, 2025, 11:02 AM GMT+5:45

File Name
Suhani Singh final submission foc.docx

File Size
4.5 MB

61 Pages

4,111 Words

22,895 Characters

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- 11 Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 3 Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
0% Publications
2% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **11** Not Cited or Quoted 4%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **3** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
0% Publications
2% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	
Islington College on 2025-08-22		2%
2	Internet	
www.coursehero.com		<1%
3	Internet	
staff.pubhealth.ku.dk		<1%
4	Submitted works	
69378 on 2015-01-22		<1%
5	Submitted works	
edubottechnologies on 2025-04-19		<1%
6	Internet	
origin.geeksforgeeks.org		<1%
7	Submitted works	
Islington College on 2025-08-18		<1%
8	Internet	
manualzz.com		<1%

Table of Contents

INTRODUCTION: **6**

AIMS: 7
OBJECTIVES: 7

DATA STRUCTURE: **8**

- PRIMITIVE DATA STRUCTURE: 8
  1. *Strings:* *8*
  2. *Integer:* *8*
  3. *Float:* *8*
  4. *Bool:* *8*
- COMPLEX DATA STRUCTURE: 9
  1. *Lists:* *9*
  2. *List of Tuples:* *10*
  3. *List Of Dicitionaries:* *10*
  4. *Functions:* *11*

TECHNOLOGIES USED: **13**

PSEUDOCODE: **16**

1. FOR OPERATION.PY: 16
2. MAIN.PY: 19
3. READ.PY: 21
4. WRITE.PY 22

ALGORITHM: **23**

FLOWCHART: **25**

PROGRAM: **27**

A. SCREENSHOTS OF THE CODE(IMPLEMENTATION) 27
B. SELLING/PURCHASE PRODUCT: 36
C. .TXT FILE: 38
D. TERMINATION OF THE PROGRAM: 39

TESTING **40**

TEST 1: TRY-EXCEPT TESTING: 40
TEST 2: PROVIDE NEGATIVE VALUE AS INPUT: 47
TEST 3: 49
TEST 4: 53

CONCLUSION: **59**

APPENDIX: **60**

LIST OF TABLES:

Table 1 Exception 1, Case 1 40
Table 2 Exception 1, Case 2 41
Table 3 Exception 2, Case 1 42
Table 4Exception 2, Case 2 43
Table 5Exception 3, Case 1 44
Table 6Exception 3, Case 2 45
Table 7Test 2 47
Table 8Test 2, Case 2 48
Table 9Test 3 49
Table 10Test 3, Case 1 50
Table 11Test 3, Case 2 51
Table 12Test 4 54
Table 13 Test 4 Case 1 55
Table 14 Test 4 Case 2 56
Table 15 Last Test 1 57
Table 16Last Test 57

INTRODUCTION:

In today's world with increasing competition every industry including the business world numerous factors plays an important role for a successful business. Among the few factors includes a clear record of the sales, products, customers and similar other information determines the progress of the business. Therefore, this report showcases a recorded detail of customer's purchase, purchased product and similar necessary data established in the record with the help of globally used programming language python through IDLE and Text Editor side by side. In the context of this scenario, the record is displayed in a form of a bill format where the information is stored in form of data and bill.

This report explains a detail of a system developed for a private wholesaler who sells shoes under the store name SpeedzWear. SpeedzWear, a private wholesale which sells or stores SKUs of different types of shoes with numerous shoe brands ranging from normal to branded according to customer's needs. The variations ranging from national shoe brands to international shoe brands in the store. The company requires a inventory of the products to understand the progress of the company. The company has its own several offers and policies. As a wholesaler, the goods are sold in large quantities leaving the company to encourage bulk buyers by providing discount on sales if the number of items is more than the predefined quantity. Discount ranges with the number of items purchases like 5% discount on sales for items more than 10. The company deeply encourages sales in domestic products by providing appealing discount offer of 7% discount on sales of domestic items more than 10 quantiles.

The company wants to keep its progress in track of the sales that updates its stock. To track the updates, a bill or invoice is generated for each transaction with the use of python and text file. Here, the text file is used to record the details of the shoes. To read the record in text file, an application is developed in python IDLE which reads the recorded data from the text file and displays the shoes available. When a purchase is made the application generates a bill or invoice for the customer with customer's purchase details respectively. After a successful purchase, the sales are recorded and updated accordingly in the stock to have clear inventory of the stock. The application should support a note/invoice for each transaction and should update in the data accordingly. The report shows when the shoe is sold, vat invoice generates as a .txt file from text editor respectively. This text file contains the details of the shoe type, shoe brand, quantity sold, name of the customer, date of transaction, the discount price and the total amount of the particular sale. The discount offers is included accordingly through the IDLE application in the invoice. Similarly, the inventory should be updated with the respective data when the company wants to restock the shoes from the vendors, A note/invoice should be generated accordingly. This report shows the process of creating and implementing the requirements mentioned.

AIMS:

The aim of this report is to explain the development of a Python-based application that keeps track of sales, inventory and updates the stock. The application is programmed to generate a billing system including the inventory, tax invoice or notes, and discounts applied based on the company's policies. The application should be able to record transactions, generate accurate bill with its respective inventory, discount policies and updating the stock.

OBJECTIVES:

The objectives of this report are:

1. To read data from the records:

The data about the goods are recorded in a text file. Through the Python program, the records are easily accessed and it reads the data. This helps to generate invoice and update the sales in then stock with a requirements like discount etc.

2. To store data:

The data is stored using list and dictionaries which is well-formatted. The respective information is added to respective label. Example: Shoe Type adds the respective shoe type information.

3. To generate Bill or Invoice:

The bill should generate with all the accurate information, details and calculation of each purchase. The bill should calculate the discount, taxes and total amount giving the customer clear idea of the payment.

4. To update the stocks:

The application should update the sales and products purchased in the stock to have clear records of products in the store

DATA STRUCTURE:

- **Primitive Data Structure:**

1. **Strings:**

String is a datatype which stores values which are quoted with " ". It includes alphabets, letters, symbols, spaces etc. String that are used are:

""Shoe Brand","Purchased Quantity", "Price", "," ,"SpeedZWear Purchase Inventory:\n" and more in the program.

2. **Integer:** An immutable data type represented by an integer number. In our program, the quantity is an integer. Similarly, integer is used indifferent loops, functions of the program.

3. **Float:** An immutable data type represented by decimal numbers with precise calculations. Here, discounts, VATetc calculations are in float.

4. **Bool:** Immutable data type represented by conditions and logic. Example:

```
shoe_found = False  # bool
for shoe in shoes:
if shoe[0] == shoe_id:
  shoe_found = True
```

Similarly, many loops used bool to execute the loop.

- **Complex Data Structure:**

1. Lists:

A list stores data as index. Using list, elements are labelled as a specific index which helps to update or retrieve data of a customer or sales.

I. Empty lists are declared which are added with data later.

```
items = []
```

II. Invoice list:

```
for line in [
    f"Shoe: {shoe[1]}({shoe[2]})",
    f"Quantity: {quantity} | Unit Price: Rs {shoe[4]}",
    f"Item Subtotal: Rs {item_total}",
    f"VAT: Rs {item_vat}",
    f"Item Total: Rs {item_final}",
    "-" * 30
]:
```

Here, the list was created to store the data in a loop to display the output in a convenient way.

III.

{
"shoe": shoe,
"quantity": quantity
}

This list helps to have multiple purchase with same data. Here, shoe is immutable whereas quantity is integer.

2. List of Tuples:

```
shoes = [
    (1, "Loafer Light", "GoldStar", 200, 1000.0, "domestic"),
    (2, "Inigo 732", "Caliber", 100, 2800.0, "domestic"),
    (3, "Lite Racer", "Adidas", 200, 7000.0, "international")
]
```

Tuples are immutable data. Here, these data are immutable.

3. List Of Dicitionaries:

```
items.append({
    "shoe": shoe,
    "quantity": quantity
})
```

This list is an ordered list in a dicitionary format: {key:value}. They are immutable so, values can be changed later.

4. Functions:

1. Save_inventory(shoes, filename="shoes.txt"):
   This function is called to save the inventory in a text file. It takes the list of tuples data and writes it in a text file.

   def read_shoes_data(filename="shoes.txt"):

2. Display_shoes(shoes):
   This function is called to gather the list of tuples and its respective elements in a neat table format. It prints each element according to its respective format.

   def display_shoes(shoes):

3. Sell_shoes(shoes)
   This function is called to allow a customer to make their preferred purchase and later update it in the files. Here, shoe is a tuple whereas quantity is integer. It also performs calculations using int and float and calls its invoice method to generate the bill.

   def sell_shoes(shoes):

4. Restock_shoes(shoes_data)
   This function is called to allow the user to restore and take the restock data in .txt file. It updates the file and dicitionaries elements accordingly.

   def restock_shoes(shoes_data):

5. Sale invoice:
   This function is called to create a general sale invoice having record of the purchase and accurate calculations. It generates .txt file too.

```
def generate_sales_invoice(customer_name, items):
```

6. Restock invoice:
   This function is called to create a general restock invoice including the record of the restock data and accurate calculations along with a .txt file.

```
def generate_restock_invoice(supplier_name, items)
```

Similarly, different functions have been called in the program. Functions help to perform a certain task and continue the progran. It helps to break a long code into managable parts. Thus, many functions were called in the program which made the code look easier and more manageable.

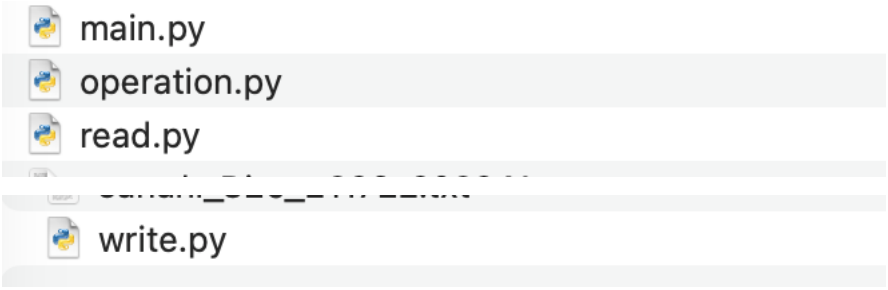
TECHNOLOGIES USED:

1. Python IDLE:

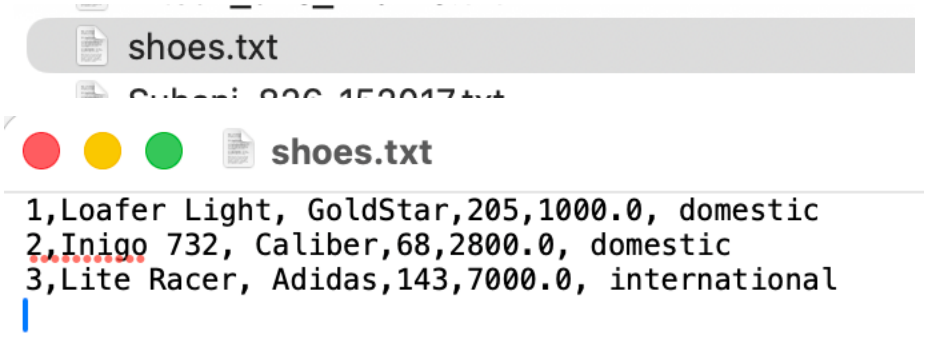


This software was used to execute the python program.

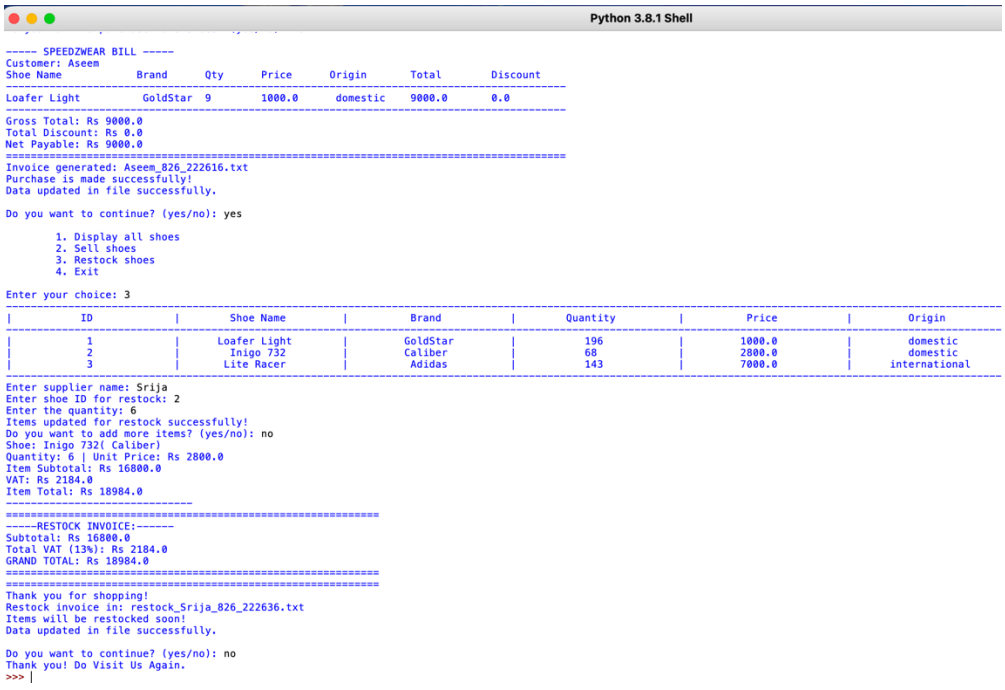
2. Four IDLE files were made to execute the given concept. This includes main.py, write.py, operation.py and read.py.



3. Text File was used to store the information of the shoes.



4. Python Shell was used to display the output of the given program.



5. Ms. Word was used for making the documentation:



PSEUDOCODE:

1. For Operation.py:

```
FUNCTION save_inventory(shoes, filename="shoes.txt")
    with open(filename, "w") as file
        FOR s in shoes
            file.write(",".join([str(value) for value in s]) + "\n")
        END FOR
END FUNCTION
FUNCTION display_shoes(shoes)
    OUTPUT("-" * 180)
    OUTPUT("|{'ID': ^25} |{'Shoe Name': ^25} |{'Brand': ^25} |{'Quantity': ^25} |{'Price': ^25}
            |{'Origin': ^25}|")
    OUTPUT("-" * 180)
    FOR shoe in shoes
        OUTPUT("|{shoe[0]: ^25} |{shoe[1]: ^25} |{shoe[2]: ^25} |{shoe[3]: ^25} |{shoe[4]: ^25}
            |{shoe[5]: ^25}|")
    END FOR
    OUTPUT("-" * 180)
END FUNCTION
FUNCTION sell_shoes(shoes)
    display_shoes(shoes)
    INITIALIZE items = []
    INITIALIZE customer_name = input("Enter customer name: ")
    WHILE TRUE
        TRY
            INITIALIZE shoe_id = int(input("Enter the shoe ID: "))
            INITIALIZE shoe_found = False
            FOR shoe in shoes
                IF shoe[0] == shoe_id
                    shoe_found = True
                    INITIALIZE quantity = int(input("No.of shoes: "))
                    IF quantity <= 0
```

```
                    OUTPUT("Quantity must be a positive number & greater than 0.")
                    CONTINUE
                END IF
                IF shoe[3] >= quantity
                    items.append({"shoe": shoe, "quantity": quantity})
                    shoe[3] -= quantity
                    OUTPUT("Items purchased successfully.")
                ELSE
                    OUTPUT("Not enough quantity available.")
                END IF
                BREAK
            END IF
        END FOR
        IF NOT shoe_found
            OUTPUT("Shoe ID not found.")
        END IF
        INITIALIZE add_more = input("Do you want to purchase more shoes? (yes/no): ")
        IF add_more != "yes"
            BREAK
        END IF
    EXCEPT ValueError
        OUTPUT("Invalid Shoe Id.")
END WHILE

OUTPUT("\n----- SPEEDZWEAR BILL -----")
OUTPUT("Customer:", customer_name)
OUTPUT("{:<20} {:<10} {:<8} {:<10} {:<12} {:<12} {:<10}".format("Shoe Name", "Brand",
  "Qty", "Price", "Origin", "Total", "Discount"))
OUTPUT("-" * 90)
INITIALIZE gross = 0
INITIALIZE total_discount = 0
INITIALIZE net = 0
FOR item in items
    INITIALIZE shoe = item["shoe"]
```

```
        INITIALIZE quantity = item["quantity"]
        INITIALIZE item_total = shoe[4] * quantity
        INITIALIZE discount = 0.0
        IF quantity > 10
            discount = 0.05
            IF shoe[5] == "domestic"
                discount = 0.07
            END IF
        END IF
        INITIALIZE discount_amount = item_total * discount
        INITIALIZE final_price = item_total - discount_amount
        gross += item_total
        total_discount += discount_amount
        net += final_price
        OUTPUT("{:<20} {:<10} {:<8} {:<10} {:<12} {:<12} {:<10}".format(shoe[1], shoe[2],
                    quantity, shoe[4], shoe[5], item_total, discount_amount))
    END FOR
    OUTPUT("-" * 90)
    OUTPUT("Gross Total: Rs", gross)
    OUTPUT("Total Discount: Rs", total_discount)
    OUTPUT("Net Payable: Rs", net)
    OUTPUT("=" * 90)
    generate_sales_invoice(customer_name, items)
    OUTPUT("Purchase is made successfully!")
END FUNCTION
```

2. Main.py:

```
FUNCTION main()
    OUTPUT("<-----SpeedzWear Wholesale Store!------>")
    WHILE True
        INITIALIZE shoes_data = read_shoes_data()
        OUTPUT("""
        1. Display all shoes
        2. Sell shoes
        3. Restock shoes
        4. Exit
        """)
        TRY
            INITIALIZE user_input = int(input("Enter your choice: "))
            IF user_input == 1
                OPERATION.display_shoes(shoes_data)
            ELSE IF user_input == 2
                OPERATION.sell_shoes(shoes_data)
                WRITE.update_data(shoes_data)
            ELSE IF user_input == 3
                OPERATION.restock_shoes(shoes_data)
                WRITE.update_data(shoes_data)
            ELSE IF user_input == 4
                OUTPUT("Thank you for using SpeedzWear system.")
                BREAK
            ELSE
                OUTPUT("Please choose from options 1-4.")
                CONTINUE
            END IF
        EXCEPT
            OUTPUT("Invalid input.")
            CONTINUE
```

```
        END TRY
        WHILE True
            INITIALIZE continue_choice = input("Do you want to continue? (yes/no): ")
            IF continue_choice in ["yes", "no"]
                BREAK
            ELSE
                OUTPUT("Please enter an option.")
            END IF
        END WHILE
        IF continue_choice != "yes"
            OUTPUT("Thank you! Do Visit Us Again.")
            BREAK
        END IF
    END WHILE
END FUNCTION

CALL main()
```

3. Read.py:

```
FUNCTION read_shoes_data(filename="shoes.txt")
    INITIALIZE shoes_data = []
    TRY
        WITH open(filename, "r") AS file
            FOR line IN file
                INITIALIZE parts = line.replace("\n", "").split(",")
                INITIALIZE shoe = [int(parts[0]), parts[1], parts[2], int(parts[3]),
float(parts[4]), parts[5]]
                shoes_data.append(shoe)
            END FOR
    EXCEPT
        OUTPUT("Error: File not found.")
    END TRY
    RETURN shoes_data
END FUNCTION
INITIALIZE data = read_shoes_data()
OUTPUT(data)
```

4. Write.py

```
FUNCTION update_data(shoes_data)
    TRY
        WITH open("shoes.txt", "w") AS file
            FOR shoe IN shoes_data
                INITIALIZE
line=f"{shoe[0]},{shoe[1]},{shoe[2]},{shoe[3]},{shoe[4]},{shoe[5]}\n"
                file.write(line)
            END FOR
            OUTPUT("Data updated in file successfully.")
    EXCEPT Exception AS e
        OUTPUT("Error, file not found: " + str(e))
    END TRY
END FUNCTION
```

ALGORITHM:

Step 1: Start python program in IDLE

Step 2: Create shoes.txt file

Step 3: Write the shoes data in the file

Step 4: Store the data in list of dicitionaires.

Step 5: Read the shoes data from python program.

Step 6: Create four python file: operation.py, main.py, write.py, read.py & wholesale.py

Step 7: Create a folder.

Step 8: Import the .txt & .py files.

Step 9: Create the necessary input in each file.

Step 10: Import the files in main.py

Step 11: Run main.py program.

Step 12: Show the shoe details in the python shell.

Step 13: Take an input from the user among

- Option 1: Display all shoes
- Option 2: Sell shoes
- Option 3: Restock shoes
- Option 4: Exit

Step 14: If input is option 1, then

- Display all the shoes details in a table.

Step 15: If input is option 2, Sell shoes then,

- Show available shoes in table
- Ask Customer's Name
- Ask shoe ID

Step 16: If the ID is valid continue

Step 17: Ask for quantity

Step 18: Check if the shoe in stock is available or not

Step 19: If the item is availabe in stock take input continue

- Else displau shoe out of stock message

Step 20: Ask customer if they want to add more product

Step 21: If yes repeat the steps again

Step 22: If no then goto next step

Step 23: Generate a bill with customer & their purchase's detail

Step 24: Save the bill as .txt file in python folder.

Step 25: Update purchased shoes in wholesale file.

Step 26: After purchase update the wholesale file.

Step 27: Display customer's purchase was a successs.

Step 28: If input is option 3, Restock shoes then,

Step 29: Display the shoes table.

Ask for Shoe ID to restock

Ask for no.of shoes for restock

Step 30: Update the stock in the wholesale file.

Step 31: Display successfully restock message.

Step 32: Goto main menu.

Step 33: Ask if the customer wants to continue.

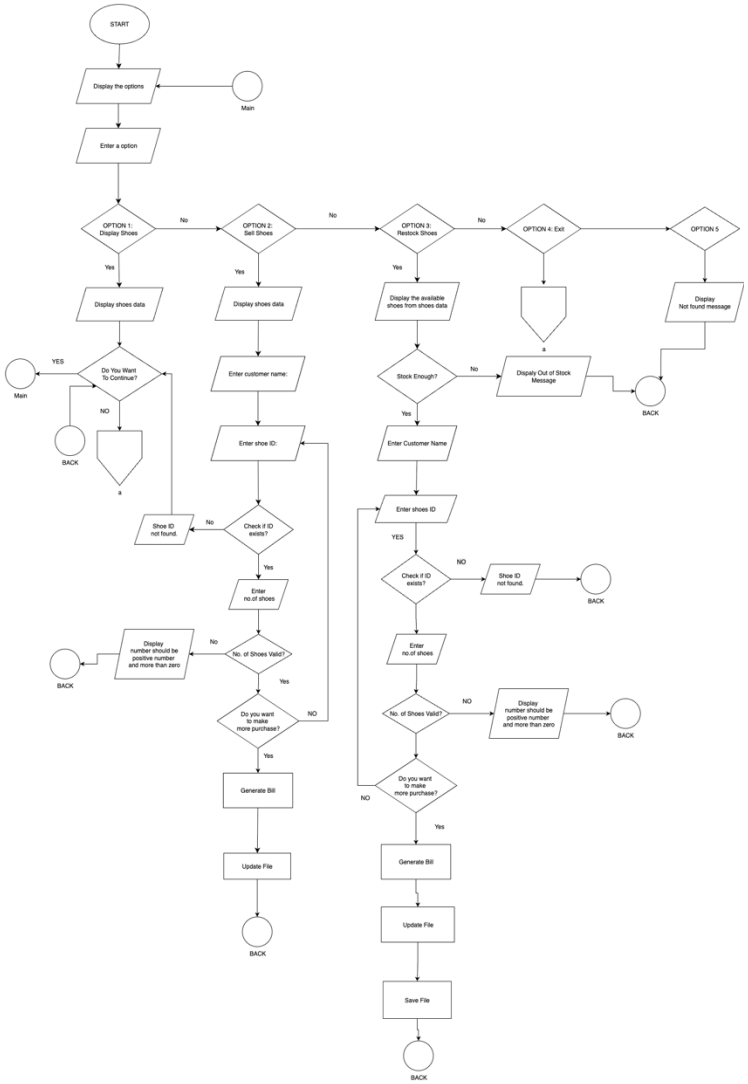
Step 34: Ask for input.

Step 35: If input is option 3, Exit then save the information and updates in the wholesale file.

Step 36: Display Thank You For Shopping message.

Step 37: End Program

FLOWCHART:


START
Display the options
Main
Enter a option
OPTION 1: Display Shoes
No
OPTION 2: Sell Shoes
No
OPTION 3: Restock Shoes
No
OPTION 4: Exit
OPTION 5
Yes
Yes
Yes
Display shoes data
Display shoes data
Display the available shoes from shoes data
a
Display Not found message
Do You Want To Continue?
YES
Main
NO
BACK
a
Enter customer name:
Stock Enough?
No
Dispaly Out of Stock Message
BACK
Yes
Enter shoe ID:
Enter Customer Name
Enter shoes ID
Shoe ID not found.
No
Check if ID exists?
YES
Check if ID exists?
NO
Shoe ID not found.
BACK
Yes
Enter no.of shoes
Enter no.of shoes
Display number should be positive number and more than zero
No
No. of Shoes Valid?
BACK
Yes
No. of Shoes Valid?
NO
Display number should be positive number and more than zero
BACK
Do you want to make more purchase?
NO
Yes
Do you want to make more purchase?
NO
Yes
Generate Bill
Generate Bill
Update File
Update File
BACK
Save File
BACK



Off page connector

Flow chart is a graphical representation of the code algorithm.

1. Oval represents start and end of the algorithm.
2. Rectangle represents the operation or process of the algorithm.
3. Diamond represents a decision point that has yes or no decisions.
4. Parallelogram represents input and output/display.
5. Arrow acts as a connector which connects each other to complete the flow chart.
6. Circle as on page connector.
7. Lastly, off page connector is used as shown above.

PROGRAM:

A. Screenshots of the code(Implementation)

i. Main.py:

*main.py - /Users

```
import datetime
from read import read_shoes_data
import operation
import write
'''calling function'''
def main():
    print("<-----SpeedzWear Wholesale Store!------>")

    while True:
        shoes_data = read_shoes_data()#reads from the shoes_data list
        print("""
        1. Display all shoes
        2. Sell shoes
        3. Restock shoes
        4. Exit
        """)

        try:
            user_input = int(input("Enter your choice: "))

            if user_input == 1:
                operation.display_shoes(shoes_data)
            elif user_input == 2:
                operation.sell_shoes(shoes_data)
                write.update_data(shoes_data)
            elif user_input == 3:
                operation.restock_shoes(shoes_data)
                write.update_data(shoes_data)
            elif user_input == 4:
                print("Thank you for using SpeedzWear system.")
                break
            else:
                print("Please choose from options 1-4.")
                continue
        except:
            print("Invalid input.")
            continue

        # Ask if user wants to continue
        while True:
            continue_choice = input("\nDo you want to continue? (yes/no): ")
            if continue_choice in ['yes', 'no']:
                break
            else:
                print("Please enter an option.")

        if continue_choice != 'yes': #for no option
            print("Thank you! Do Visit Us Again.")
            break

main()
```

ii. Operation.py:

operation.py - /Users/hrithiksingh/Desktop/Python/operation.py (3.8.1)

```
import datetime
import read
import write

# Inventory(list of tuples)
shoes=[
    (1, "Loafer Light", "GoldStar", 200, 1000.0, "domestic"),
    (2, "Inigo 732", "Caliber", 100, 2800.0, "domestic"),
    (3, "Lite Racer", "Adidas", 200, 7000.0, "international")
    ]

'''calling function to read txt file'''
def save_inventory(shoes, filename="shoes.txt"):
    with open(filename,"w") as file:
        for s in shoes:
            file.write(",".join([str(value)for value in s]) +"\n")

def display_shoes(shoes):#to display shoes in table
    print("-" * 180)
    print(f"|{'ID' : ^25} |{'Shoe Name': ^25} |{'Brand': ^25} |{'Quantity': ^25} |{'Price': ^25} |{'Origin': ^25}|")
    print("-" * 180)
    for shoe in shoes:
        print(f"|{shoe[0]: ^25} |{shoe[1]: ^25} |{shoe[2]: ^25} |{shoe[3]: ^25} |{shoe[4]: ^25} |{shoe[5]: ^25}|")
    print("-"* 180)

'''calling function for sell part'''
def sell_shoes(shoes):
    display_shoes(shoes)
    items = []#declaring empty list
    customer_name = input("Enter customer name: ")

    while True:
        try:
            shoe_id = int(input("Enter the shoe ID: "))

            # Check if shoe ID exists
            shoe_found = False
            for shoe in shoes:
                if shoe[0] == shoe_id:
                    shoe_found = True
                    quantity = int(input("No.of shoes: "))
                    if quantity <= 0:
                        print("Quantity must be a positive number & greater than 0.")
                        continue

                    if shoe[3] >= quantity:
                        items.append({
                            "shoe": shoe,
                            "quantity": quantity
                        })
                        shoe[3] -= quantity
                        print("Items purchased successfully.")
                    else:
                        print("Not enough quantity available.")
                    break
```

The operation.py contains the commands for operation the shell carries out.

Here, different functions, tuples,lists, dicitionaries are implemented.

operation.py - /Users/hrithiksingh/Desktop/Python/operation.py (3.8.1)

```
    print("Net Payable: Rs", net)
    print("=" * 90)

    #generate invoice(for 0 items too)
    generate_sales_invoice(customer_name, items)
    print("Purchase is made successfully!")

'''calling function for restock option'''
def restock_shoes(shoes_data):
    display_shoes(shoes_data)
    items = []
    supplier_name = input("Enter supplier name: ")

    while True:
        try:
            shoe_id = int(input("Enter shoe ID for restock: "))

            shoe_found = False
            for shoe in shoes_data:#check if the shoe Id exists
                if shoe[0] == shoe_id:
                    shoe_found = True
                    quantity = int(input("Enter the quantity: "))
                    if quantity <= 0:
                        print("Quantity must be a positive number and greater than 0.")
                        continue

                    items.append({
                        "shoe": shoe,
                        "quantity": quantity
                    })
                    shoe[3] += quantity
                    print("Items updated for restock successfully!")
                    break

            if not shoe_found:
                print("Shoe not found in the stock.")

            #To add more items
            add_more = input("Do you want to add more items? (yes/no): ")
            if add_more!= 'yes':
                break#again starts from beginning

        except:
            print("Invalid Input.")

    generate_restock_invoice(supplier_name, items)
    print("Items will be restocked soon!")#message after input

def generate_sales_invoice(customer_name, items):
    '''generate bill including tax, invoice and VAT'''
    today = datetime.datetime.today()
    year = today.year
    month = today.month
    day = today.day
    hour = today.hour
```

This program generates the sales and the restock invoice ensuring accurate calculations. It ensures to give proper exception output message as well.

operation.py - /Users/hrithiksingh/Desktop/Python/operation.py (3.8.1)

```
    generate_restock_invoice(supplier_name, items)
    print("Items will be restocked soon!")#message after input


def generate_sales_invoice(customer_name, items):
    '''to generate bill including tax, invoice and VAT'''
    today = datetime.datetime.today()
    year = today.year
    month = today.month
    day = today.day
    hour = today.hour
    minute = today.minute
    second = today.second


    filename = f"{customer_name}_{month}{day}_{hour}{minute}{second}.txt"

    #initialise
    total_discount = 0
    total_vat = 0
    grand_total = 0
    subtotal = 0

    with open(filename, "w") as file:
        file.write("=" * 96 + "\n")
        file.write("SPEEDZWEAR VAT INVOICE\n")
        file.write("=" * 96 + "\n")
        file.write("Customer: " + customer_name + "\n")
        file.write("Date: " + str(year) + "-" + str(month) + "-" + str(day) + " " + str(hour) + ":" + str(minute) + ":" + str(second) + "\n")

        file.write("-" * 96  + "\n")
        file.write("Items Purchased:\n")
        file.write("-" * 96 + "\n")

        if not items:
            file.write("No items purchased.\n")
        else:
            for item in items:
                shoe = item["shoe"]
                quantity = item["quantity"]
                item_total = shoe[4] * quantity
                discount = 0.0
                #for greater than 10 quantity, discount is applied
                if quantity > 10:
                    discount = 0.05
                    if shoe[5] == "domestic":
                        discount = 0.07
                '''discount amount calculation'''
                discount_amount = item_total * discount
                item_subtotal = item_total - discount_amount
                item_vat = item_subtotal * 0.13
                item_final = item_subtotal + item_vat

                total_discount += discount_amount
                total_vat += item_vat
                grand_total += item_final
```

operation.py - /Users/hrithiksingh/Desktop/Python/operation.py (3.8.1)

```
                grand_total += item_final
                subtotal += item_total
                #invoice with discount amount
                file.write("Shoe: " + shoe[1] + " (" + shoe[2] + ")\n")
                file.write("Quantity: " + str(quantity) + " | Unit Price: Rs " + str(shoe[4]) + "\n")
                file.write("Item Total: Rs " + str(item_total) + "\n")
                if discount_amount > 0:
                    file.write("Discount: " + str(discount*100) + "% (Rs " + str(discount_amount) + ")\n")
                file.write("VAT: Rs " + str(item_vat) + "\n")
                file.write("Item Final: Rs " + str(item_final) + "\n")
                file.write("-" * 30 + "\n")

        file.write("=" * 60 + "\n")
        file.write("SUMMARY:\n")
        file.write("Subtotal: Rs " + str(subtotal) + "\n")
        file.write("Total Discount: Rs " + str(total_discount) + "\n")
        file.write("Total VAT (13%): Rs " + str(total_vat) + "\n")
        file.write("GRAND TOTAL: Rs " + str(grand_total) + "\n")
        file.write("=" * 60 + "\n")
        file.write("Thank you! Do Visit Us Again.\n")
        file.write("=" * 60 + "\n")

    print("Invoice generated: " + filename)

'''calling fucntion to generate restock invoice'''
def generate_restock_invoice(supplier_name, items):
    today = datetime.datetime.today()
    year = today.year
    month = today.month
    day = today.day
    hour = today.hour
    minute = today.minute
    second = today.second

    #restock file
    filename = f"restock_{supplier_name}_{month}{day}_{hour}{minute}{second}.txt"

    total_vat = 0
    grand_total = 0
    subtotal = 0

    with open(filename, "w") as file:
        file.write("=" * 60 + "\n")
        file.write("SPEEDZWEAR RESTOCK INVOICE\n")
        file.write("=" * 60 + "\n")
        file.write("Supplier: " + supplier_name + "\n")
        file.write("Date: " + str(year) + "-" + str(month) + "-" + str(day) + " " + str(hour) + ":" + str(minute) + ":" + str(second) + "\n")
        file.write("-" * 60 + "\n")
        file.write("Items Restocked:\n")
        file.write("-" * 60 + "\n")

        if not items:
            file.write("Sorry, no items for restock!\n")
            print("Sorry,no items for restock!")
        else:
            for item in items:
```

The invoice is printed in .txt file and the shell itself.

operation.py - /Users/hrithiksingh/Desktop/Python/operation.py (3.8.1)

```
total_vat = 0
grand_total = 0
subtotal = 0

with open(filename, "w") as file:
    file.write("=" * 60 + "\n")
    file.write("SPEEDZWEAR RESTOCK INVOICE\n")
    file.write("=" * 60 + "\n")
    file.write("Supplier: " + supplier_name + "\n")
    file.write("Date: " + str(year) + "-" + str(month) + "-" + str(day) + " " + str(hour) + ":" + str(minute) + ":" + str(second) + "\n")
    file.write("-" * 60 + "\n")
    file.write("Items Restocked:\n")
    file.write("-" * 60 + "\n")

    if not items:#checks the item
        file.write("Sorry, no items for restock!\n")
        print("Sorry,no items for restock!")
    else:
        for item in items:
            shoe = item["shoe"]
            quantity = item["quantity"]
            '''calculates amount for bill'''
            item_total = shoe[4] * quantity
            item_vat = item_total * 0.13
            item_final = item_total + item_vat

            total_vat += item_vat
            grand_total += item_final
            subtotal += item_total

            for line in[
                f"Shoe: {shoe[1]}({shoe[2]})",
                f"Quantity: {quantity} | Unit Price: Rs {shoe[4]}",
                f"Item Subtotal: Rs {item_total}",
                f"VAT: Rs {item_vat}",
                f"Item Total: Rs {item_final}",
                "-" * 30
            ]:
                file.write(line+"\n")
                print(line)
    for line in[
         "=" * 60,
    "-----RESTOCK INVOICE:------",
    f"Subtotal: Rs {subtotal}",
    f"Total VAT (13%): Rs {total_vat}",
    f"GRAND TOTAL: Rs {grand_total}",
    "=" * 60,
    "=" * 60
    ]:
        file.write(line+"\n")
        print(line)

print("Thank you for shopping!")

print("Restock invoice in: " + filename)
```

To display the invoice in the shell, list has been used where the information is stored that is later displayed.

iii. Read.py:

read.py - /Users/hrithiksingh/Desktop/Python/read.py (3.8.1)

```
def read_shoes_data(filename="shoes.txt"):#reads the shoe data
    shoes_data = []#empty list
    try:
        with open("shoes.txt","r") as file:
            for line in file:#line is an object for the file
                parts = line.replace("\n","").split(",")
                shoe = [int(parts[0]),
                        (parts[1]),
                        (parts[2]),
                        int(parts[3]),
                        float(parts[4]), parts[5]]
                shoes_data.append(shoe)#adds shoe list the empty list
    except:
        print("Error: File not found.")
    return shoes_data
data = read_shoes_data()
print(data)
```

This program reads the data from shoes.txt file and adds the content in a well structured form as instructed.

iv. Write.py:

*write.py - /Users/hrithiksingh/

```
def update_data(shoes_data):'''calling function to update data'''
    try:
        with open("shoes.txt", "w") as file:#open .txt file
            for shoe in shoes_data:
                line = f"{shoe[0]},{shoe[1]},{shoe[2]},{shoe[3]},{shoe[4]},{shoe[5]}\n"
                file.write(line)#update.txt file content
        print("Data updated in file successfully.")
    except Exception as e:
        print(f"Error, file not found: {e}")
```

Read.py loads the data, Write.py saves the data, and Operation. py performs all inventory operations

OUTPUT:

Python 3.8.1 Shell

```
Python 3.8.1 (v3.8.1:1b293b6006, Dec 18 2019, 14:08:53)
[Clang 6.0 (clang-600.0.57)] on darwin
Type "help", "copyright", "credits" or "license()" for more information.
>>>
============= RESTART: /Users/hrithiksingh/Desktop/Python/main.py =============
[[1, 'Loafer Light', ' GoldStar', 205, 1000.0, ' domestic'], [2, 'Inigo 732', ' Caliber', 68, 2800.0, ' domestic'], [3, 'Lite Racer', ' Adidas', 143, 7000.0, '
<-----SpeedzWear Wholesale Store!------>

        1. Display all shoes
        2. Sell shoes
        3. Restock shoes
        4. Exit

Enter your choice: 1
-------------------------------------------------------------------------------------------------------------------------------------------------------
|          ID           |        Shoe Name        |         Brand          |       Quantity        |         Price         |        Origin         |
-------------------------------------------------------------------------------------------------------------------------------------------------------
|           1           |      Loafer Light       |         GoldStar       |          205          |         1000.0        |        domestic       |
|           2           |        Inigo 732        |         Caliber        |          68           |         2800.0        |        domestic       |
|           3           |       Lite Racer        |          Adidas        |          143          |         7000.0        |      international    |
-------------------------------------------------------------------------------------------------------------------------------------------------------

Do you want to continue? (yes/no): 2
Please enter an option.

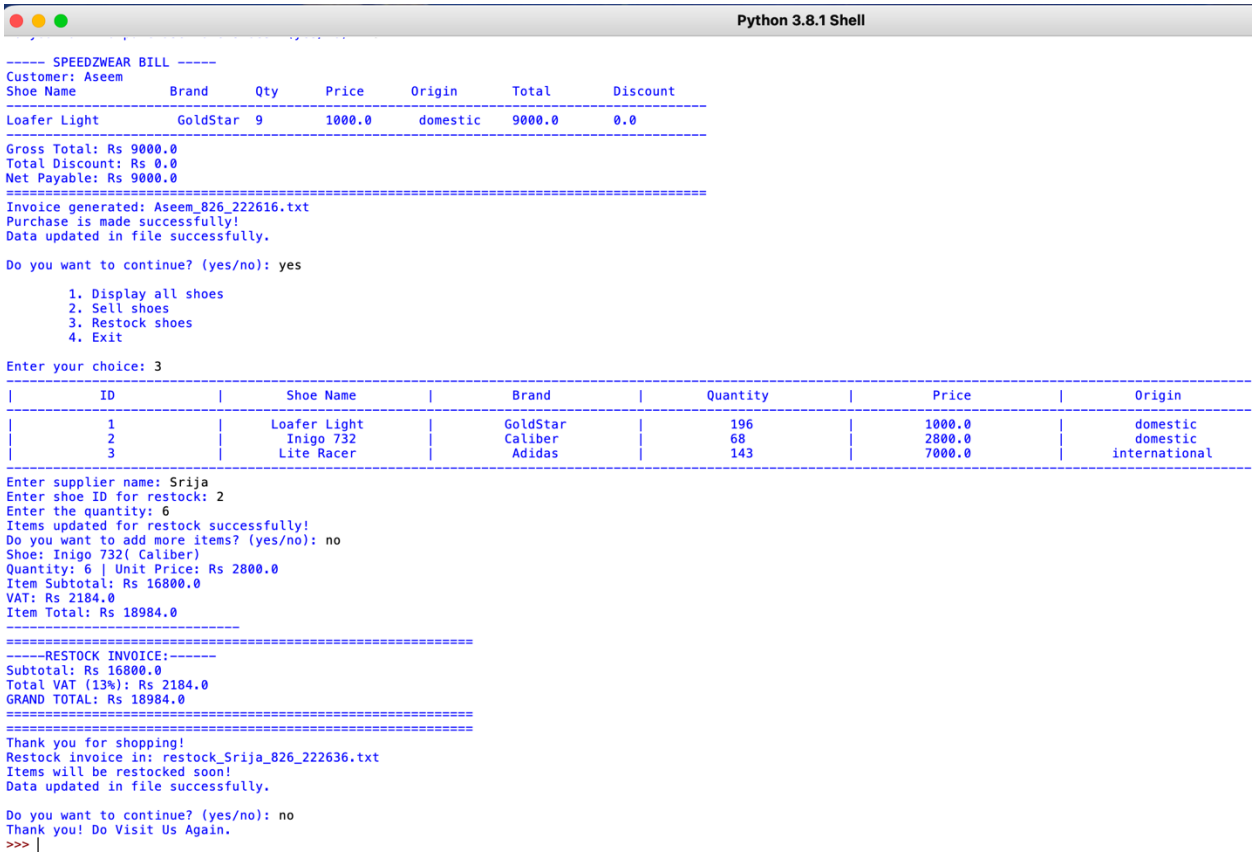
Do you want to continue? (yes/no): yes

        1. Display all shoes
        2. Sell shoes
        3. Restock shoes
        4. Exit

Enter your choice: 2
-------------------------------------------------------------------------------------------------------------------------------------------------------
|          ID           |        Shoe Name        |         Brand          |       Quantity        |         Price         |        Origin         |
-------------------------------------------------------------------------------------------------------------------------------------------------------
|           1           |      Loafer Light       |         GoldStar       |          205          |         1000.0        |        domestic       |
|           2           |        Inigo 732        |         Caliber        |          68           |         2800.0        |        domestic       |
|           3           |       Lite Racer        |          Adidas        |          143          |         7000.0        |      international    |
-------------------------------------------------------------------------------------------------------------------------------------------------------
Enter customer name: Aseem
Enter the shoe ID: 1
No.of shoes: 9
Items purchased successfully.
Do you want to purchase more shoes? (yes/no): no

----- SPEEDZWEAR BILL -----
Customer: Aseem
Shoe Name            Brand      Qty      Price      Origin       Total        Discount
-------------------------------------------------------------------------------------
Loafer Light          GoldStar  9        1000.0     domestic     9000.0       0.0
-------------------------------------------------------------------------------------
Gross Total: Rs 9000.0
Total Discount: Rs 0.0
Net Payable: Rs 9000.0
=====================================================================================
```

The output displays the available shoes and customer purchasing an item. The output displays the invoice of the purchased item too.



Here, the user restocks an item from a supplier. The data is filled and the invoice is generated.

B. Selling/purchase product:

The user first enters option 2.

```
Do you want to continue? (yes/no): yes

        1. Display all shoes
        2. Sell shoes
        3. Restock shoes
        4. Exit

Enter your choice: 
```

The following table is displayed. Here, the user is asked to fill the details.

```
Enter your choice: 2
-------------------------------------------------------------------------------------------------------------------------------------------------------
|         ID          |       Shoe Name       |         Brand         |       Quantity        |         Price         |        Origin         |
-------------------------------------------------------------------------------------------------------------------------------------------------------
|          1          |     Loafer Light      |        GoldStar       |          180          |        1000.0         |        domestic       |
|          2          |       Inigo 732       |        Caliber        |          92           |        2800.0         |        domestic       |
|          3          |      Lite Racer       |         Adidas        |          104          |        7000.0         |     international     |
-------------------------------------------------------------------------------------------------------------------------------------------------------
Enter customer name: Suhani
Enter the shoe ID: 3
No.of shoes: 36
Items purchased successfully.
Do you want to purchase more shoes? (yes/no):
```

When the details is filled it asks if they want to make more purchase.

ACTUAL OUTPUT:

```
Enter customer name: Suhani
Enter the shoe ID: 3
No.of shoes: 36
Items purchased successfully.
Do you want to purchase more shoes? (yes/no): yes
Enter the shoe ID: 2
No.of shoes: 92
Items purchased successfully.
Do you want to purchase more shoes? (yes/no): no
```

Purchase invoice:

```
Do you want to purchase more shoes? (yes/no): no

----- SPEEDZWEAR BILL -----
Customer: Suhani
Shoe Name            Brand      Qty      Price      Origin       Total        Discount
-------------------------------------------------------------------------------------------
Lite Racer            Adidas    36       7000.0      international 252000.0     12600.0
Inigo 732             Caliber   92       2800.0      domestic    257600.0     12880.0
-------------------------------------------------------------------------------------------
Gross Total: Rs 509600.0
Total Discount: Rs 25480.0
Net Payable: Rs 484120.0
===========================================================================================
Invoice generated: Suhani_826_152017.txt
Purchase is made successfully!
Data updated in file successfully.

Do you want to continue? (yes/no):
```

C. .txt file:

Suhani_826_152017.txt

```
============================================================
Customer: Suhani
Date: 2025-8-26 15:20:17
------------------------------------------------------------
Items Purchased:
------------------------------------------------------------
Shoe: Lite Racer ( Adidas)
Quantity: 36 | Unit Price: Rs 7000.0
Item Total: Rs 252000.0
Discount: 5.0% (Rs 12600.0)
VAT: Rs 31122.0
Item Final: Rs 270522.0
------------------------------
Shoe: Inigo 732 ( Caliber)
Quantity: 92 | Unit Price: Rs 2800.0
Item Total: Rs 257600.0
Discount: 5.0% (Rs 12880.0)
VAT: Rs 31813.600000000002
Item Final: Rs 276533.6
------------------------------
============================================================
SUMMARY:
Subtotal: Rs 509600.0
Total Discount: Rs 25480.0
Total VAT (13%): Rs 62935.600000000006
GRAND TOTAL: Rs 547055.6
============================================================
Thank you! Do Visit Us Again.
============================================================
```

main.py	Today at 10:11 PM	2 KB	Python Script
operation.py	Today at 9:46 PM	10 KB	Python Script
read.py	Today at 9:33 PM	648 bytes	Python Script
restock_Divya_826_203241.txt	Today at 8:32 PM	938 bytes	Plain Text
restock_Suhani_826_154559.txt	Today at 3:45 PM	795 bytes	Plain Text
Ritesh_826_215719.txt	Today at 9:57 PM	788 bytes	Plain Text
shoes.txt	Today at 9:57 PM	134 bytes	Plain Text
Suhani_826_152017.txt	Today at 3:20 PM	1 KB	Plain Text
suhani_826_211722.txt	Today at 9:17 PM	788 bytes	Plain Text
write.py	22 Aug 2025 at 10:40 AM	364 bytes	Python Script

D. Termination of the program:

For no option:

```
Do you want to continue? (yes/no): no
Thank you! Do Visit Us Again.
>>>
```

For exit option:

```
        1. Display all shoes
        2. Sell shoes
        3. Restock shoes
        4. Exit

Enter your choice: 4
Thank you for using SpeedzWear system.
>>>
```

TESTING

TEST 1: TRY-EXCEPT TESTING:

- **Choosing non-existing option:**

S.NO.	EXCEPTION 1	RESULT
1.	**Objective:**	To check the exception of entering non existing option.
2.	**Action**	Entering an exception option i.e. "5".
3.	**Expected Output**	A message to choose option from 1-4 should display.
4.	**Actual Output**	The message is displayed.
5.	**RESULT**	The exception handling was successful.

Table 1 Exception 1, Case 1

ACTUAL OUTPUT:

```
<-----SpeedzWear Wholesale Store!------>

        1. Display all shoes
        2. Sell shoes
        3. Restock shoes
        4. Exit

Enter your choice: 5
Please choose from options 1-4.
```

Entering non-integer input:

S.NO.	EXCEPTION 1, CASE 2	RESULT
1.	**Objective:**	To check the exception of invalid input.
2.	**Action**	Enters alphabets(invalid input) in the choice.
3.	**Expected Output**	An invalid input message should display.
4.	**Actual Output**	The message is displayed.
5.	**RESULT**	The exception handling was successful.

Table 2 Exception 1, Case 2

ACTUAL OUTPUT:

```
Please choose from options 1-4.

        1. Display all shoes
        2. Sell shoes
        3. Restock shoes
        4. Exit

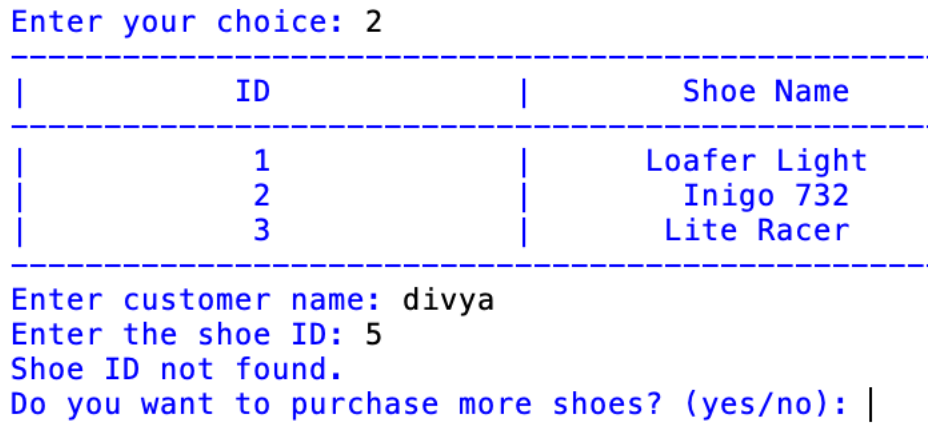
Enter your choice: AB
Invalid input.
```

- **Shoe ID Exception:**

S.NO.	EXCEPTION 2, CASE 1	RESULT
6.	**Objective:**	To check the exception of entering only existing shoe IDs.
7.	**Action**	Entering an exception shoe ID i.e. "5".
8.	**Expected Output**	A message should display saying shoe ID not found.
9.	**Actual Output**	The message is displayed.
10.	**RESULT**	The exception handling was successful.

Table 3 Exception 2, Case 1

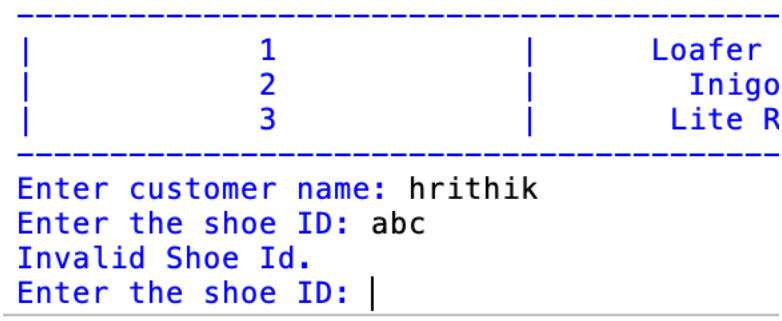
ACTUAL OUTPUT:



S.NO.	EXCEPTION 2, Case 2	RESULT
6.	**Objective:**	To check the exception in shoe ID of invalid input.
7.	**Action**	Enters alphabets(invalid input) in ID.
8.	**Expected Output**	An error message should display.
9.	**Actual Output**	The message is displayed.
10.	**RESULT**	The exception handling was successful.

Table 4Exception 2, Case 2

ACTUAL OUTPUT:

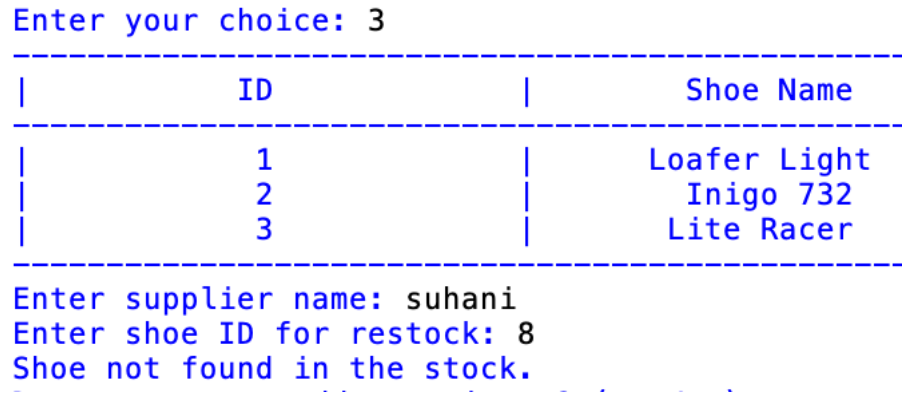


- **Restock Exception**:

S.NO.	EXCEPTION 3, Case 1	RESULT
1.	**Objective:**	To check if the shoes is available for restock.
2.	**Action**	Entering a non existing shoe ID.
3.	**Expected Output**	A message should display saying shoe not found in the stock.
4.	**Actual Output**	The message is displayed.
5.	**RESULT**	The exception handling was successful.

Table 5Exception 3, Case 1

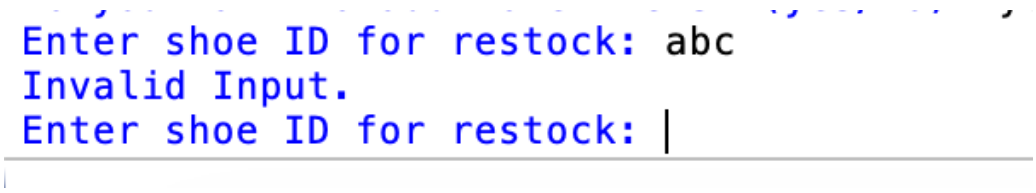
ACTUAL OUTPUT:



S.NO.	EXCEPTION 3, Case 2	RESULT
11.	**Objective:**	To check the exception of invalid input.
12.	**Action**	Enters alphabets(invalid input) in ID.
13.	**Expected Output**	An error message should display.
14.	**Actual Output**	The message is displayed.
15.	**RESULT**	The exception handling was successful.

Table 6Exception 3, Case 2

ACTUAL OUTPUT:



- **EXCEPTION 4:**

```
try:
    with open("shoes.txt","r") as file:
        for line in file:#line is an object for the file
            parts = line.replace("\n","").split(",")
            shoe = [int(parts[0]),
                    (parts[1]),
                    (parts[2]),#str datatype
                    int(parts[3]),
                    float(parts[4]), parts[5]]
            shoes_data.append(shoe)
    except:
        print("Error: File not found.")
```

This exception is used to read the file from shoes.txt if the file is found. It generates the data from .txt file and displays according to shoe list. If file not found it displays error message.

- **EXCEPTION 5:**

```
try:
    with open("shoes.txt", "w") as file:
        for shoe in shoes_data:
            line = f"{shoe[0]},{shoe[1]},{shoe[2]},{shoe[3]},{shoe[4]},{shoe[5]}\n"
            file.write(line)
    print("Data updated in file successfully.")
    except Exception as e:
        print(f"Error, file not found: {e}")
```

TEST 2: PROVIDE NEGATIVE VALUE AS INPUT:

S.NO.	TEST 2	RESULT
1.	**Objective:**	To test the non-existent value.
2.	**Action**	The option "-9" was entered.
3.	**Expected Output**	Message should display saying to add positive number.
4.	**Actual Output**	The message is displayed.
5.	**RESULT**	The test was successful.

Table 7Test 2

ACTUAL OUTPUT:

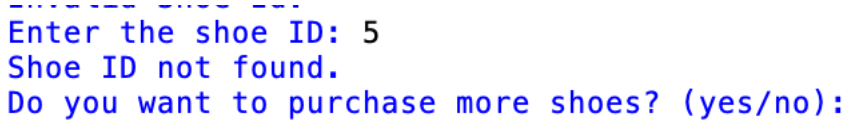
```
Enter your choice: 2
-----------------------------------------------------
|           ID             |        Shoe Name         |
-----------------------------------------------------
|            1             |      Loafer Light        |
|            2             |        Inigo 732         |
|            3             |       Lite Racer         |
-----------------------------------------------------
Enter customer name: Ritesh
Enter the shoe ID: 2
No.of shoes: -9
Quantity must be a positive number & greater than 0.
Do you want to purchase more shoes? (yes/no):
```

- **PROVIDE NON-EXISTENT VALUE:**

S.NO.	TEST 2, Case 2	RESULT
6.	**Objective:**	To test the non-existent value.
7.	**Action**	The option "5" was entered.
8.	**Expected Output**	Shoe ID not found message should display.
9.	**Actual Output**	The message is displayed.
10.	**RESULT**	The test was successful.

Table 8Test 2, Case 2

ACTUAL OUTPUT:



TEST 3: PURCHASE PROCESS

S.NO.	TEST 3	RESULT
11.	**Objective:**	To test the process of purchase.
12.	**Action**	The option "2" was entered.
13.	**Expected Output**	The available shoes should be displayed. Then, the user & their purchase details should be asked to fill.
14.	**Actual Output**	Asks the user to fill the details. The successful purchase message is shown and it asks the user if they want to purchase more.
15.	**RESULT**	The test was successful.

Table 9Test 3

ACTUAL OUTPUT:

```
Enter your choice: 2
-------------------------------------------------------------------------------------------------------------------------------------------------------
|          ID          |        Shoe Name        |         Brand          |        Quantity        |         Price          |        Origin          |
-------------------------------------------------------------------------------------------------------------------------------------------------------
|           1          |      Loafer Light       |        GoldStar        |          180           |         1000.0         |        domestic        |
|           2          |        Inigo 732        |        Caliber         |           92           |         2800.0         |        domestic        |
|           3          |       Lite Racer        |         Adidas         |          104           |         7000.0         |     international      |
-------------------------------------------------------------------------------------------------------------------------------------------------------
Enter customer name: Suhani
Enter the shoe ID: 3
No.of shoes: 36
Items purchased successfully.
Do you want to purchase more shoes? (yes/no):
```

CASE 1:

S.NO.	CASE 1	RESULT
1.	**Objective:**	Testing "yes" option.
2.	**Action**	The user enters "yes".
3.	**Expected Output**	Program should ask the details for the purchase user wants to add. Then, it should ask the user if they want to make furthur more purchase.
4.	**Actual Output**	Asks the user to fill the details & displays a message saying items purchased sucessfully.
5.	**RESULT**	The test was successful.

Table 10Test 3, Case 1

ACTUAL OUTPUT:

```
Enter customer name: Suhani
Enter the shoe ID: 3
No.of shoes: 36
Items purchased successfully.
Do you want to purchase more shoes? (yes/no): yes
Enter the shoe ID: 2
No.of shoes: 92
Items purchased successfully.
Do you want to purchase more shoes? (yes/no): no
```

CASE 2:

S.NO.	CASE 2	RESULT
1.	**Objective:**	To implement "no" option.
2.	**Action**	The user enters "no".
3.	**Expected Output**	The invoice or bill of the purchased item should display. The location of invoice's .txt file should show with a message and the data should be updated.
4.	**Actual Output**	The bill is generated where the amount is calculated with respective discount. It saves in the .txt file and updates the data.
5.	**RESULT**	The test was successful.

Table 11Test 3, Case 2

ACTUAL OUTPUT:

```
Do you want to purchase more shoes? (yes/no): no

----- SPEEDZWEAR BILL -----
Customer: Suhani
Shoe Name            Brand      Qty      Price      Origin       Total        Discount
-------------------------------------------------------------------------------------
Lite Racer            Adidas    36       7000.0      international 252000.0     12600.0
Inigo 732             Caliber   92       2800.0      domestic    257600.0     12880.0
-------------------------------------------------------------------------------------
Gross Total: Rs 509600.0
Total Discount: Rs 25480.0
Net Payable: Rs 484120.0
=====================================================================================
Invoice generated: Suhani_826_152017.txt
Purchase is made successfully!
Data updated in file successfully.

Do you want to continue? (yes/no):
```

TERMINAL OUTPUT:

```
============================================================
SPEEDZWEAR VAT INVOICE
============================================================
Customer: Suhani
Date: 2025-8-26 15:20:17
------------------------------------------------------------
Items Purchased:
------------------------------------------------------------
Shoe: Lite Racer ( Adidas)
Quantity: 36 | Unit Price: Rs 7000.0
Item Total: Rs 252000.0
Discount: 5.0% (Rs 12600.0)
VAT: Rs 31122.0
Item Final: Rs 270522.0
----------------------------
Shoe: Inigo 732 ( Caliber)
Quantity: 92 | Unit Price: Rs 2800.0
Item Total: Rs 257600.0
Discount: 5.0% (Rs 12880.0)
VAT: Rs 31813.600000000002
Item Final: Rs 276533.6
----------------------------
============================================================
SUMMARY:
Subtotal: Rs 509600.0
Total Discount: Rs 25480.0
Total VAT (13%): Rs 62935.600000000006
GRAND TOTAL: Rs 547055.6
============================================================
Thank you! Do Visit Us Again.
============================================================
```

TEST 4: UPDATED DATA

The Original:

```
Loafer Light, GoldStar, 200, 1000, domestic
Inigo 732, Caliber, 100, 2800, domestic
Lite Racer, Adidas, 200, 7000, international
```

Updated:

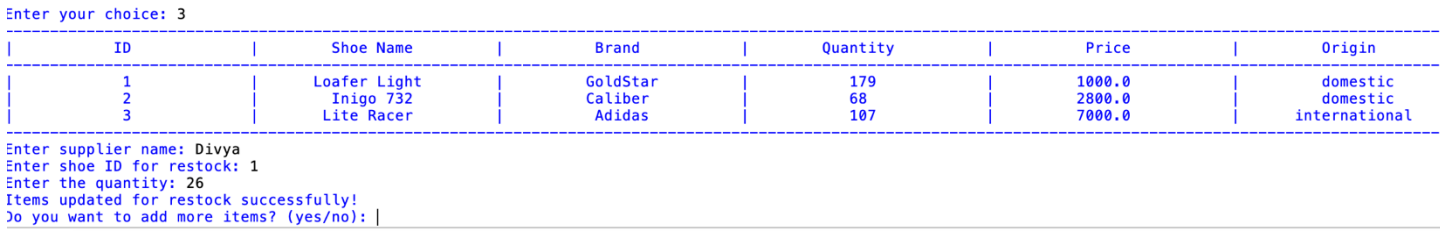
shoes.txt

```
1,Loafer Light, GoldStar,179,1000.0, domestic
2,Inigo 732, Caliber,68,2800.0, domestic
3,Lite Racer, Adidas,107,7000.0, international
```

S.NO.	TEST 4	RESULT
1.	**Objective:**	To test option 3 i.e. restock shoes.
2.	**Action**	The user enter "3".
3.	**Expected Output**	The shoes table should display the existing quantity. Program should ask the details of the supplier and about the restock.
4.	**Actual Output**	The table of shoes is displayed. The data for restock is asked. Then, the data is updated fot the restock and asks if user wants to add another item.
5.	**RESULT**	The test was successful.

Table 12Test 4

ACTUAL OUTPUT:



CASE 1:

S.NO.	CASE 1	RESULT
1.	**Objective:**	To implement "yes" option.
2.	**Action**	The user enters "yes".
3.	**Expected Output**	Program should ask the details for the purchase user wants to add for restock. Then, it should ask the user if they want to add more items for purchase.
4.	**Actual Output**	Asks the user to fill the details & displays a message saying items updated for restock sucessfully. Then, it asks if the user wants to add more items for restock again.
5.	**RESULT**	The test was successful.

Table 13 Test 4 Case 1

ACTUAL OUTPUT:

```
Do you want to add more items? (yes/no): yes
Enter shoe ID for restock: 3
Enter the quantity: 36
Items updated for restock successfully!
Do you want to add more items? (yes/no):
```

CASE 2:

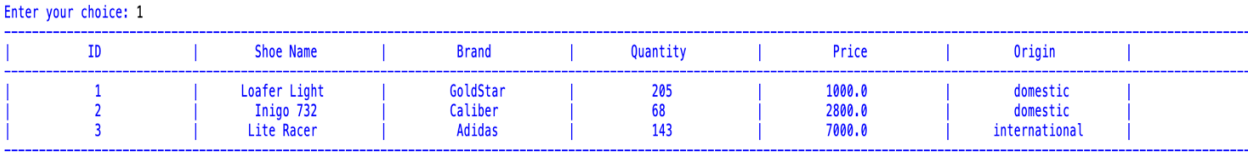
S.NO.	CASE 2	RESULT
1.	**Objective:**	To implement "no" option.
2.	**Action**	The user enters "no".
3.	**Expected Output**	The invoice or bill of the restocked item should display. The location of invoice's .txt file should show with a message and the data should be updated.
4.	**Actual Output**	The bill is generated where the amount is calculated with respective VAT. It saves in the .txt file displaying a message and updates the data.
5.	**RESULT**	The test was successful.

Table 14 Test 4 Case 2

ACTUAL OUTPUT:

```
Shoe: Loafer Light( GoldStar)
Quantity: 26 | Unit Price: Rs 1000.0
Item Subtotal: Rs 26000.0
VAT: Rs 3380.0
Item Total: Rs 29380.0
------------------------------
Shoe: Lite Racer( Adidas)
Quantity: 36 | Unit Price: Rs 7000.0
Item Subtotal: Rs 252000.0
VAT: Rs 32760.0
Item Total: Rs 284760.0
------------------------------
============================================================
-----RESTOCK INVOICE:------
Subtotal: Rs 278000.0
Total VAT (13%): Rs 36140.0
GRAND TOTAL: Rs 314140.0
============================================================
============================================================
Thank you for shopping!
Restock invoice in: restock_Divya_826_203241.txt
Items will be restocked soon!
Data updated in file successfully.
```

The stock is been updated next time the data is displayed:



Test Case For Shoe ID: 1

S.No.	CASE (Shoe ID:1)	Value
1.	Shoe ID	1
2.	Previous Quantity	179
3.	Restocked Quantity	26
4.	Updated Quantity	179+26=205
5.	Expected Updated Quantity	205
6.	Conclusion	The test was successful.

Table 15 Last Test 1

Test Case For Shoe ID: 3

S.No.	CASE (Shoe ID:1)	Value
1.	Shoe ID	3
2.	Previous Quantity	107
3.	Restocked Quantity	36
4.	Updated Quantity	107+36=143
5.	Expected Updated Quantity	143
6.	Conclusion	The test was successful.

Table 16Last Test 2

Updated .txt file:

shoes.txt

```
1,Loafer Light, GoldStar,205,1000.0, domestic
2,Inigo 732, Caliber,68,2800.0, domestic
3,Lite Racer, Adidas,143,7000.0, international
```

.txt invoice:

```
============================================================
SPEEDZWEAR RESTOCK INVOICE
============================================================
Supplier: Divya
Date: 2025-8-26 20:32:41
------------------------------------------------------------
Items Restocked:
------------------------------------------------------------
Shoe: Loafer Light( GoldStar)
Quantity: 26 | Unit Price: Rs 1000.0
Item Subtotal: Rs 26000.0
VAT: Rs 3380.0
Item Total: Rs 29380.0
------------------------------
Shoe: Lite Racer( Adidas)
Quantity: 36 | Unit Price: Rs 7000.0
Item Subtotal: Rs 252000.0
VAT: Rs 32760.0
Item Total: Rs 284760.0
------------------------------
============================================================
-----RESTOCK INVOICE:------
Subtotal: Rs 278000.0
Total VAT (13%): Rs 36140.0
GRAND TOTAL: Rs 314140.0
============================================================
============================================================
```

CONCLUSION:

This report shows the operation of SpeedZWear Wholesale Store data through Python application successfully. It implements a complete inventory management for billing percent. It ensures to read the available data, store the purchased quantity with the customer's details and update the data. This facilities are applied using lists, dicitionaries, tuples and plenty other python concepts. Applying these concepts, the system allows the user to view the available shoes, purchase preffered amount of shoes with accurate discount and VAT calculations. These information is included in the invoice.

The invoice is specially designed such that it gives special discounts. It provides 7% discount for domestic and 5% for international. This discount is applied when the purchased content. Invoice updates the data as per the amount which are sold or restocked. Additionally, multiple functions that are used allows the program to be scalable, maintainable, and easy to understand.

Testings confirms the functionality of the code and its implementation. Multiple exception handlings are used to avoid any unnecessary crash. E xception handling handles the error without crashing. The project also illustrates the practical use of primitive and complex data structures, including strings, integers, floats, booleans, lists, tuples, and dictionaries, giving a better understanding of these concepts.

Overall, this coursework highlights the effective integration of python programming, file handling, data structures, and logical operations to develop a real-world wholesale store application, providing both functional and clear documentation for clear vision.

APPENDIX:

- **Main.py**

```
import datetime
from read import read_shoes_data
import operation
import write
"'calling function"'
def main():
    print("<-----SpeedzWear Wholesale Store!------>")

    while True:
        shoes_data = read_shoes_data()#reads from the shoes_data list
        print("""
        1. Display all shoes
        2. Sell shoes
        3. Restock shoes
        4. Exit
        """)

        try:
            user_input = int(input("Enter your choice: "))

            if user_input == 1:
                operation.display_shoes(shoes_data)
            elif user_input == 2:
                operation.sell_shoes(shoes_data)
                write.update_data(shoes_data)
            elif user_input == 3:
                operation.restock_shoes(shoes_data)
                write.update_data(shoes_data)
            elif user_input == 4:
```

```
            print("Thank you for using SpeedzWear system.")
            break
        else:
            print("Please choose from options 1-4.")
            continue
    except:
        print("Invalid input.")
        continue

    # Ask if user wants to continue
    while True:
        continue_choice = input("\nDo you want to continue? (yes/no): ")
        if continue_choice in ['yes', 'no']:
            break
        else:
            print("Please enter an option.")

    if continue_choice != 'yes': #for no option
        print("Thank you! Do Visit Us Again.")
        break

main()
```

- **Operation.py**

```
import datetime
import read
import write

# Inventory(list of tuples)
shoes=[
    (1, "Loafer Light", "GoldStar", 200, 1000.0, "domestic"),
    (2, "Inigo 732", "Caliber", 100, 2800.0, "domestic"),
    (3, "Lite Racer", "Adidas", 200, 7000.0, "international")
    ]

"'calling function to read txt file'"
def save_inventory(shoes, filename="shoes.txt"):
    with open(filename,"w") as file:
        for s in shoes:
            file.write(",".join([str(value)for value in s]) +"\n")

def display_shoes(shoes):#to display shoes in table
    print("-" * 180)
    print(f"|{'ID' : ^25} |{'Shoe Name': ^25} |{'Brand': ^25} |{'Quantity': ^25} |{'Price': ^25}
|{'Origin': ^25}|")
    print("-" * 180)
    for shoe in shoes:
        print(f"|{shoe[0]: ^25} |{shoe[1]: ^25} |{shoe[2]: ^25} |{shoe[3]: ^25} |{shoe[4]: ^25}
|{shoe[5]: ^25}|")
    print("-"* 180)

"'calling function for sell part'"
def sell_shoes(shoes):
    display_shoes(shoes)
    items = []#declaring empty list
    customer_name = input("Enter customer name: ")
```

```
while True:
    try:
        shoe_id = int(input("Enter the shoe ID: "))

        # Check if shoe ID exists
        shoe_found = False
        for shoe in shoes:
            if shoe[0] == shoe_id:
                shoe_found = True
                quantity = int(input("No.of shoes: "))
                if quantity <= 0:
                    print("Quantity must be a positive number & greater than 0.")
                    continue

                if shoe[3] >= quantity:
                    items.append({
                        "shoe": shoe,
                        "quantity": quantity
                    })
                    shoe[3] -= quantity
                    print("Items purchased successfully.")
                else:
                    print("Not enough quantity available.")
                break

        if not shoe_found:
            print("Shoe ID not found.")

        #Ask to add more purchase
        add_more = input("Do you want to purchase more shoes? (yes/no): ")
        if add_more!= 'yes':
            break

    except ValueError:
```

```
        print("Invalid Shoe Id.")

    #Display PurchaseBill
print("\n----- SPEEDZWEAR BILL -----")
print("Customer:", customer_name)
print("{:<20} {:<10} {:<8} {:<10} {:<12} {:<12} {:<10}".format(
   "Shoe Name", "Brand", "Qty", "Price", "Origin", "Total", "Discount"))
print("-" * 90)

gross = 0
total_discount = 0
net = 0

for item in items:
   shoe = item["shoe"]
   quantity = item["quantity"]

   item_total = shoe[4] * quantity
   discount = 0.0
   if quantity > 10:#discount condition
      discount = 0.05
      if shoe[5] == "domestic":
         discount = 0.07

   discount_amount = item_total * discount
   final_price = item_total - discount_amount

   gross += item_total
   total_discount += discount_amount
   net += final_price

   print("{:<20} {:<10} {:<8} {:<10} {:<12} {:<12} {:<10}".format(
      shoe[1], shoe[2], quantity, shoe[4], shoe[5], item_total, discount_amount))
```

```
    print("-" * 90)
    print("Gross Total: Rs", gross)
    print("Total Discount: Rs", total_discount)
    print("Net Payable: Rs", net)
    print("=" * 90)

    #generate invoice(for 0 items too)
    generate_sales_invoice(customer_name, items)
    print("Purchase is made successfully!")

'''calling function for restock option'''
def restock_shoes(shoes_data):
    display_shoes(shoes_data)
    items = []
    supplier_name = input("Enter supplier name: ")

    while True:
        try:
            shoe_id = int(input("Enter shoe ID for restock: "))

            shoe_found = False
            for shoe in shoes_data:#check if the shoe Id exists
                if shoe[0] == shoe_id:
                    shoe_found = True
                    quantity = int(input("Enter the quantity: "))
                    if quantity <= 0:
                        print("Quantity must be a positive number and greater than 0.")
                        continue

                    items.append({
                        "shoe": shoe,
                        "quantity": quantity
                    })
                    shoe[3] += quantity
```

```
                print("Items updated for restock successfully!")
                break

            if not shoe_found:
                print("Shoe not found in the stock.")

            #To add more items
            add_more = input("Do you want to add more items? (yes/no): ")
            if add_more!= 'yes':
                break#again starts from beginning

        except:
            print("Invalid Input.")

    generate_restock_invoice(supplier_name, items)
    print("Items will be restocked soon!")#message after input


def generate_sales_invoice(customer_name, items):
    '''to generate bill including tax, invoice and VAT'''
    today = datetime.datetime.today()
    year = today.year
    month = today.month
    day = today.day
    hour = today.hour
    minute = today.minute
    second = today.second


    filename = f"{customer_name}_{month}{day}_{hour}{minute}{second}.txt"

    #initialise
    total_discount = 0
    total_vat = 0
```

```
    grand_total = 0
    subtotal = 0

    with open(filename, "w") as file:
        file.write("=" * 96 + "\n")
        file.write("SPEEDZWEAR VAT INVOICE\n")
        file.write("=" * 96 + "\n")
        file.write("Customer: " + customer_name + "\n")
        file.write("Date: " + str(year) + "-" + str(month) + "-" + str(day) + " " + str(hour) + ":" +
str(minute) + ":" + str(second) + "\n")

        file.write("-" * 96  + "\n")
        file.write("Items Purchased:\n")
        file.write("-" * 96 + "\n")

        if not items:
            file.write("No items purchased.\n")
        else:
            for item in items:
                shoe = item["shoe"]
                quantity = item["quantity"]
                item_total = shoe[4] * quantity
                discount = 0.0
                #for greater than 10 quantity, discount is applied
                if quantity > 10:
                    discount = 0.05
                    if shoe[5] == "domestic":
                        discount = 0.07
                '''discount amount calculation'''
                discount_amount = item_total * discount
                item_subtotal = item_total - discount_amount
                item_vat = item_subtotal * 0.13
                item_final = item_subtotal + item_vat
```

```
            total_discount += discount_amount
            total_vat += item_vat
            grand_total += item_final
            subtotal += item_total
            #invoice with discount amount
            file.write("Shoe: " + shoe[1] + " (" + shoe[2] + ")\n")
            file.write("Quantity: " + str(quantity) + " | Unit Price: Rs " + str(shoe[4]) + "\n")
            file.write("Item Total: Rs " + str(item_total) + "\n")
            if discount_amount > 0:
                file.write("Discount: " + str(discount*100) + "% (Rs " + str(discount_amount) +
")\n")
            file.write("VAT: Rs " + str(item_vat) + "\n")
            file.write("Item Final: Rs " + str(item_final) + "\n")
            file.write("-" * 30 + "\n")

        file.write("=" * 60 + "\n")
        file.write("SUMMARY:\n")
        file.write("Subtotal: Rs " + str(subtotal) + "\n")
        file.write("Total Discount: Rs " + str(total_discount) + "\n")
        file.write("Total VAT (13%): Rs " + str(total_vat) + "\n")
        file.write("GRAND TOTAL: Rs " + str(grand_total) + "\n")
        file.write("=" * 60 + "\n")
        file.write("Thank you! Do Visit Us Again.\n")
        file.write("=" * 60 + "\n")

    print("Invoice generated: " + filename)

'''calling fucntion to generate restock invoice'''
def generate_restock_invoice(supplier_name, items):
    today = datetime.datetime.today()
    year = today.year
    month = today.month
    day = today.day
    hour = today.hour
```

```
    minute = today.minute
    second = today.second

    #restock file
    filename = f"restock_{supplier_name}_{month}{day}_{hour}{minute}{second}.txt"

    total_vat = 0
    grand_total = 0
    subtotal = 0

    with open(filename, "w") as file:
        file.write("=" * 60 + "\n")
        file.write("SPEEDZWEAR RESTOCK INVOICE\n")
        file.write("=" * 60 + "\n")
        file.write("Supplier: " + supplier_name + "\n")
        file.write("Date: " + str(year) + "-" + str(month) + "-" + str(day) + " " + str(hour) + ":" +
str(minute) + ":" + str(second) + "\n")
        file.write("-" * 60 + "\n")
        file.write("Items Restocked:\n")
        file.write("-" * 60 + "\n")

        if not items:#checks the item
            file.write("Sorry, no items for restock!\n")
            print("Sorry,no items for restock!")
        else:
            for item in items:
                shoe = item["shoe"]
                quantity = item["quantity"]
                '''calculates amount for bill'''
                item_total = shoe[4] * quantity
                item_vat = item_total * 0.13
                item_final = item_total + item_vat

                total_vat += item_vat
```

```
            grand_total += item_final
            subtotal += item_total

            for line in[
                f"Shoe: {shoe[1]}({shoe[2]})",
                f"Quantity: {quantity} | Unit Price: Rs {shoe[4]}",
                f"Item Subtotal: Rs {item_total}",
                f"VAT: Rs {item_vat}",
                f"Item Total: Rs {item_final}",
                "-" * 30
            ]:
                file.write(line+"\n")
                print(line)
    for line in[
        "=" * 60,
    "-----RESTOCK INVOICE:------",
    f"Subtotal: Rs {subtotal}",
    f"Total VAT (13%): Rs {total_vat}",
    f"GRAND TOTAL: Rs {grand_total}",
    "=" * 60,
    "=" * 60
    ]:
        file.write(line+"\n")
        print(line)

print("Thank you for shopping!")

print("Restock invoice in: " + filename)
```

- **Read.py:**

```
def read_shoes_data(filename="shoes.txt"):#reads the shoe data
    shoes_data = []#empty list
    try:
        with open("shoes.txt","r") as file:
            for line in file:#line is an object for the file
                parts = line.replace("\n","").split(",")
                shoe = [int(parts[0]),
                        (parts[1]),
                        (parts[2]),
                        int(parts[3]),
                        float(parts[4]), parts[5]]
                shoes_data.append(shoe)#adds shoe list the empty list
    except:
        print("Error: File not found.")
    return shoes_data
data = read_shoes_data()
print(data)
```

- **Write.py:**

```
def update_data(shoes_data):
    try:
        with open("shoes.txt", "w") as file:
            for shoe in shoes_data:
                line = f"{shoe[0]},{shoe[1]},{shoe[2]},{shoe[3]},{shoe[4]},{shoe[5]}\n"
                file.write(line)
        print("Data updated in file successfully.")
    except Exception as e:
        print(f"Error, file not found: {e}")
```